

# Node.js Concepts Overview

---



complete node js concept

Here's a **complete Node.js roadmap (beginner → advanced → interview-ready)** explained in a structured way so you can *learn + build + revise easily*.

## 1. What is Node.js?

---

- Node.js is a **runtime environment** to run JavaScript outside the browser.
- Built on **Chrome's V8 engine**
- Uses **non-blocking (asynchronous) architecture**

 Example:

JavaScript

```
console.log("Start"); setTimeout(() => {  
  console.log("Async Task");  
}, 2000); console.log("End");
```

📌 Output:

```
Start  
End  
Async Task
```

## ⚙️ 2. How Node.js Works

---

### ◆ Event Loop + Non-blocking I/O

---

- Single-threaded but handles multiple requests
- Uses:
  - Call Stack
  - Event Queue
  - Event Loop

👉 Important concepts:

- Synchronous vs Asynchronous
- Blocking vs Non-blocking

## 3. Modules in Node.js

---

### ◆ Types:

---

#### 1. Core Modules

`fs, http, path, os`

#### 2. Local Modules

#### 3. Third-party Modules (npm)

👉 Example:

JavaScript

```
const fs = require('fs'); fs.writeFileSync("test.txt", "Hello Node");
```

## 4. File System (fs Module)

---

### ◆ Sync vs Async

---

JavaScript

```
// Sync
fs.writeFileSync("file.txt", "Hello"); // Async
fs.writeFile("file.txt", "Hello", (err) => {
  if (err) throw err;
});
```

## 5. HTTP Module (Creating Server)

---

JavaScript

```
const http = require('http'); const server = http.createServer((req, res) => {  
  res.write("Hello World");  
  res.end();  
}); server.listen(3000);
```

## 6. npm (Node Package Manager)

---

Install packages:

Bash

```
npm init -y  
npm install express
```

## 7. Express.js (Most Important)

---

👉 Simplifies backend development

JavaScript

```
const express = require('express');
const app = express(); app.get('/', (req, res) => {
  res.send("Home Page");
}); app.listen(3000);
```

## 8. Middleware

---

👉 Functions between request & response

JavaScript

```
app.use((req, res, next) => {
  console.log("Middleware running");
  next();
});
```



## 9. Routing

---

JavaScript

```
app.get('/user', (req, res) => {  
  res.send("User Data");  
});
```



## 10. Request & Response

---

### ◆ Request:

---

- `req.params`
- `req.query`
- `req.body`

### ◆ Response:

---

- `res.send()`
- `res.json()`
- `res.status()`



## 11. REST API

---

### CRUD:

---

- GET → Read
- POST → Create
- PUT → Update
- DELETE → Delete

JavaScript

```
app.post('/user', (req, res) => {  
  res.send("User Created");  
});
```

## 12. Database (Very Important)

---

### ◆ MongoDB (MERN stack)

---

Use with **Mongoose**

JavaScript

```
const mongoose =  
require('mongoose'); mongoose.connect("mongodb://localhost:27017/test");
```

## 13. Authentication

---

### ◆ Methods:

---

- JWT (JSON Web Token)
- Sessions
- Cookies

JavaScript

```
const jwt = require('jsonwebtoken'); const token = jwt.sign({ id: 1 }, "secret");
```

## 14. Async Programming

---

- ◆ **Callback**

---

- ◆ **Promise**

---

- ◆ **Async/Await**

---

JavaScript

```
async function getData() {  
  return "Data";  
}
```



## 15. Error Handling

---

JavaScript

```
app.use((err, req, res, next) => {  
  res.status(500).send(err.message);  
});
```

## 16. MVC Architecture

---

- Model → Database
- View → UI
- Controller → Logic

## 17. File Upload (Multer)

---

JavaScript

```
const multer = require('multer');
```

## 18. WebSockets (Real-time)

---

- Chat apps
- Live updates

Use:

Bash

```
npm install socket.io
```

## 19. Testing

---

- Jest
- Mocha

## 20. Deployment

---

- Render
- Vercel
- AWS
- DigitalOcean

## 21. Advanced Topics

---

- Streams & Buffers
- Clustering
- Worker Threads
- Rate Limiting
- Caching (Redis)
- Security (Helmet, CORS)
- Scaling APIs

## 22. Interview Important Topics

---

- ✓ Event Loop
- ✓ Async vs Sync
- ✓ Streams
- ✓ Middleware
- ✓ REST API
- ✓ JWT
- ✓ Error Handling
- ✓ Express Routing
- ✓ MongoDB Integration

## 23. Real Projects You Should Build

---

1. Blog API
2. Authentication System
3. E-commerce Backend
4. Chat App (Socket.io)
5. URL Shortener
6. Payment Integration

## Final Learning Path

---

👉 Follow this order:

1. JS Basics
2. Node Core
3. Express
4. APIs
5. Database
6. Auth
7. Projects
8. Advanced

If you want, I can:

- ✓ Give **full notes (PDF style)**
- ✓ Create **interview questions + answers**

✅ Guide you step-by-step to build your **Zerodha Clone backend**

Just tell me 👍

node file

It looks like you're asking about “**Node file**” (how to run or work with a file in Node.js). Let's make it super clear 👉

## 📁 What is a Node.js File?

---

A **Node.js file** is just a JavaScript file with `.js` extension that runs using Node.

👉 Example:

```
app.js
server.js
index.js
```

## ▶ How to Run a Node File

---

### ✅ Step 1: Install Node.js

---

Check:

```
Bash
```

```
node -v
```

## ✓ Step 2: Create a File

---

Bash

```
touch app.js
```

(or manually create `app.js`)

## ✓ Step 3: Write Code

---

JavaScript

```
console.log("Hello Node");
```

## ✓ Step 4: Run File

---

Bash

```
node app.js
```

 Output:

```
Hello Node
```

## Your Previous Error (IMPORTANT)

---

You wrote:

```
node --version  (inside browser console)
```

 That's wrong because:

- `node` commands run in **terminal (CMD / PowerShell / VS Code terminal)**
- NOT inside browser console

## Types of Node Files

---

### 1. Simple Script

---

```
JavaScript
```

```
console.log("Running file");
```

## 2. Server File

---

JavaScript

```
const http = require('http'); http.createServer((req, res) => {  
  res.end("Server running");  
}).listen(3000);
```

Run:

Bash

```
node server.js
```

## 3. Module File

---

👉 file1.js

JavaScript

```
exports.add = (a, b) => a + b;
```

👉 file2.js

JavaScript

```
const math = require('./file1');  
console.log(math.add(2, 3));
```

## Auto Restart (Very Useful)

---

Use **nodemon**:

Bash

```
npm install -g nodemon  
nodemon app.js
```



## Common Errors (Fix)

---

### Error:

---

```
node is not defined
```

 Reason: Running in browser

 Fix:

- ✓ Open terminal
- ✓ Run:

```
Bash
```

```
node app.js
```

## Pro Tips

---

- Always run Node files in **terminal**
- Use `console.log()` to debug
- Use `nodemon` for development

- Keep main file as:

```
index.js OR app.js
```

If you want next step, I can teach you:

👉 How to connect Node file with **Express server**

👉 Or build a **complete backend project step-by-step** 🚀

```
node repl
```

Great—let's understand **Node REPL** clearly (this is very important for basics + interviews).

## 🧠 What is Node REPL?

---

**REPL = Read – Eval – Print – Loop**

👉 It is an **interactive Node.js shell** where you can write and execute JavaScript code line-by-line.

## ▶ How to Start Node REPL

---

Open **terminal / CMD / VS Code terminal** and type:

```
Bash
```

```
node
```

📌 You will see:

```
>
```

That `>` means REPL is active 

## REPL Working (Step-by-step)

---

### ◆ 1. Read

---

Takes input from user

```
> 2 + 3
```

### ◆ 2. Eval

---

Evaluates the expression

### ◆ 3. Print

---

Outputs result

```
5
```

### ◆ 4. Loop

---

Waits for next input



## Basic Examples

---

JavaScript

```
> 10 * 5
50
> "Hello " + "Node"
'Hello Node'
> let a = 10
undefined
> a + 5
15
```



## Using Variables

---

JavaScript

```
> var x = 100
undefined
> x
100
```



## Special REPL Commands

---

| Command                    | Use               |
|----------------------------|-------------------|
| <code>.help</code>         | Show all commands |
| <code>.exit</code>         | Exit REPL         |
| <code>.clear</code>        | Clear screen      |
| <code>.save file.js</code> | Save session      |

| Command                    | Use       |
|----------------------------|-----------|
| <code>.load file.js</code> | Load file |

## Multiline Code

---

JavaScript

```
> function add(a, b) {  
... return a + b;  
... }  
undefined > add(2, 3)  
5
```

## Access Node Modules

---

JavaScript

```
> const fs = require('fs');  
undefined > fs.existsSync('app.js')  
true
```

## Common Mistake (Your Case)

---

 Running in browser console:

```
node
```

👉 Error:

```
node is not defined
```

✓ Correct:

- Open **terminal**
- Then run:

```
Bash
```

```
node
```



## Exit REPL

---

```
JavaScript
```

```
.exit
```

or press:

```
Ctrl + C (twice)
```



## Why Use REPL?

---

- ✓ Quick testing
- ✓ Debugging
- ✓ Learning JavaScript
- ✓ Checking Node features



## Interview Question

---

👉 Q: What is REPL in Node.js?

👉 Answer:

Node REPL is an interactive shell that reads user input, evaluates it, prints the result, and loops continuously.

If you want next level 🔥

I can teach you:

- 👉 Difference between **REPL vs Node file**
- 👉 Internal working of REPL
- 👉 Advanced tricks (like auto-complete, underscore `_` variable, etc.)

process in node

Let's understand **process** in **Node.js** in a clear, practical way (very important for interviews + real projects).

## What is **process** in Node.js?

---

👉 **process** is a **global object**

👉 It gives information about the **current Node.js process (running program)**

📌 You don't need to import it:

```
JavaScript
```

```
console.log(process);
```

## 1. Process Information

---

### ◆ **Platform & Version**

---

```
JavaScript
```

```
console.log(process.platform); // win32 / linux  
console.log(process.version);  // Node version
```



## ◆ Current Directory

---

JavaScript

```
console.log(process.cwd()); // current working directory
console.log(process.pid);   // process ID
```

## 📦 2. Command Line Arguments (**process.argv**)

---

👉 Used to take input from terminal

Bash

```
node app.js Aamir 21
```

JavaScript

```
console.log(process.argv);
```

📌 Output:

JavaScript

```
[  
  'node path',  
  'app.js path',  
  'Aamir',  
  '21'  
]
```

👉 Access values:

JavaScript

```
const name = process.argv[2];  
console.log("Hello", name);
```

### 3. Environment Variables (**process.env**)

---

👉 Used for config (very important in backend)

JavaScript

```
console.log(process.env.PORT);
```

### ◆ Example:

---

Bash

```
PORT=5000 node app.js
```

JavaScript

```
const port = process.env.PORT || 3000;  
console.log(port);
```

## 4. Exit Process

---

JavaScript

```
process.exit();
```

👉 With status:

JavaScript

```
process.exit(1); // error  
process.exit(0); // success
```



## 5. Events in Process

---

### ◆ Exit Event

---

JavaScript

```
process.on('exit', () => {  
  console.log("Process ended");  
});
```

### ◆ Uncaught Error

---

JavaScript

```
process.on('uncaughtException', (err) => {  
  console.log("Error:", err.message);  
});
```



## 6. Process Timing

---

JavaScript

```
console.time("test"); for (let i = 0; i < 1000000; i++) {} console.timeEnd("test");
```



## 7. Memory Usage

---

JavaScript

```
console.log(process.memoryUsage());
```



## 8. Next Tick (Important)

---

JavaScript

```
process.nextTick(() => {  
  console.log("Next Tick");  
});
```

👉 Runs before:

- `setTimeout`
- `setImmediate`

## 🔄 Example (Execution Order)

---

JavaScript

```
console.log("Start"); process.nextTick(() => {  
  console.log("Next Tick");  
}); setTimeout(() => {  
  console.log("Timeout");  
}, 0); console.log("End");
```

📌 Output:

```
Start  
End  
Next Tick  
Timeout
```

## ⚠ Common Mistake

---

❌ Running in browser:

```
JavaScript
```

```
process is not defined
```

👉 Reason:

`process` exists only in **Node.js**, not browser

## 🎯 Interview Questions

---

👉 **Q1: What is process in Node?**

👉 A global object that provides information and control over the current Node.js process.

👉 **Q2: What is `process.argv`?**

👉 It stores command-line arguments.

👉 **Q3: Difference between `process.nextTick()` and `setTimeout()`?**

👉 `nextTick()` runs immediately after current operation, before event loop.

## 🚀 Real Use Cases

---

- ✓ CLI tools
- ✓ Environment config
- ✓ Debugging
- ✓ Server port setup
- ✓ Error handling

If you want next step 🔥

I can teach you:

- 👉 **Event Loop (deep concept)**
- 👉 OR build a **CLI tool using `process.argv`**
- 👉 OR connect this with your **Express project**

export in files

Let's understand **export in Node.js files** step-by-step (this is very important for modular coding + interviews).



## Why Export in Node.js?

---

👉 We use **export** to share code (functions, variables, objects) from one file to another.

### 1. CommonJS (Default in Node.js)

---

Node.js mainly uses **CommonJS**:

- `module.exports`
- `exports`

#### ◆ Method 1: `module.exports`

---

#### 👉 file1.js

---

JavaScript

```
const add = (a, b) => a + b;  
const sub = (a, b) => a - b; module.exports = { add, sub };
```

#### 👉 file2.js

---

JavaScript

```
const math = require('./file1'); console.log(math.add(5, 3)); // 8  
console.log(math.sub(5, 3)); // 2
```

## ◆ Method 2: exports

---

JavaScript

```
exports.add = (a, b) => a + b;  
exports.sub = (a, b) => a - b;
```

👉 Import same way:

JavaScript

```
const math = require('./file1');
```

## ⚠ Important Difference

---

✗ Wrong:

JavaScript

```
exports = { add, sub };    // ✗ will NOT work
```

✓ Correct:

JavaScript

```
module.exports = { add, sub };
```

👉 Reason:

`exports` is just a reference to `module.exports`

## ◆ Export Single Function

---

👉 file1.js

---

JavaScript

```
module.exports = function(name) {  
  return "Hello " + name;  
};
```

JavaScript

```
const greet = require('./file1'); console.log(greet("Aamir"));
```

## ◆ Export Object

---

JavaScript

```
module.exports = {  
  name: "Aamir",  
  age: 21  
};
```

## ◆ Export Class

---

JavaScript

```
class User {
  constructor(name) {
    this.name = name;
  }
}
module.exports = User;
```

## 2. ES Modules (Modern Way)

---

👉 Use `import/export`

⚙️ **Enable:**

---

Add in `package.json`:

JSON

```
{
  "type": "module"
}
```

## 👉 file1.js

---

JavaScript

```
export const add = (a, b) => a + b;  
export const sub = (a, b) => a - b;
```

## 👉 file2.js

---

JavaScript

```
import { add, sub } from './file1.js'; console.log(add(2, 3));
```

## ◆ Default Export

---

JavaScript

```
export default function greet() {  
  console.log("Hello");  
}
```

👉 Import:

JavaScript

```
import greet from './file1.js';
```

## ⚖️ CommonJS vs ES Modules

---

| Feature         | CommonJS       | ES Modules   |
|-----------------|----------------|--------------|
| Syntax          | require        | import       |
| Export          | module.exports | export       |
| Sync/Async      | Sync           | Async        |
| Default in Node | Yes            | Needs config |

## Common Errors

---

### Error:

---

```
JavaScript
```

```
Cannot use import outside module
```

### ✓ Fix:

- Add `"type": "module"` in package.json  
OR
- Use `.mjs` extension

## Interview Questions

---

- 👉 Difference between `exports` and `module.exports`
- 👉 Can we use both `require` and `import` together?
- 👉 What is default export?

## Real Use Case Example

---

👉 math.js

```
JavaScript
```

```
exports.add = (a, b) => a + b;
```



👉 app.js

JavaScript

```
const { add } = require('./math'); console.log(add(10, 5));
```

## 🧠 Pro Tip

---

👉 Always prefer:

JavaScript

```
module.exports = {}
```

👉 for clean and scalable code

If you want next 🔥

I can teach you:

👉 **require() deep concept**

👉 OR build a **modular backend project (like Zerodha clone)**

👉 OR give **advanced Node.js interview questions**

export in directories

Great question—this is **very important for clean project structure** 👉

## 📁 Export in Directories (Node.js)

---

👉 When you have **multiple files inside a folder**, you can export them using a single entry file (usually **index.js**).

📌 This helps you import everything using **one line only**.

## 🧠 Basic Idea

---

Instead of:

JavaScript

```
const user = require('./controllers/user');  
const product = require('./controllers/product');
```

👉 You do:

JavaScript

```
const controllers = require('./controllers');
```

## Folder Structure

---

```
project/
├── controllers/
│   ├── user.js
│   ├── product.js
│   └── index.js    👉 main export file
└── app.js
```

### ◆ Step 1: Create Files

---

#### 👉 user.js

---

JavaScript

```
exports.getUser = () => {
  return "User Data";
};
```

## 👉 product.js

---

JavaScript

```
exports.getProduct = () => {  
  return "Product Data";  
};
```

### ◆ Step 2: Export from Directory (**index.js**)

---

👉 controllers/index.js

JavaScript

```
const user = require('./user');  
const product = require('./product'); module.exports = {  
  user,  
  product  
};
```

### ◆ Step 3: Import in Main File

---

👉 app.js

JavaScript

```
const controllers = require('./controllers'); console.log(controllers.user.getUser());  
console.log(controllers.product.getProduct());
```

## **Shortcut (Best Practice)**

---

👉 Use object destructuring:

JavaScript

```
const { user, product } = require('./controllers'); user.getUser();  
product.getProduct();
```

## **Advanced Pattern (Cleaner)**

---

👉 controllers/index.js

JavaScript

```
module.exports = {  
  ...require('./user'),  
  ...require('./product')  
};
```

👉 Now use directly:

JavaScript

```
const { getUser, getProduct } = require('./controllers');  
getUser();  
getProduct();
```

## ES Modules Version

---

👉 controllers/index.js

JavaScript

```
export * from './user.js';  
export * from './product.js';
```

👉 app.js

JavaScript

```
import { getUser, getProduct } from './controllers/index.js';
```

## ⚠ Important Notes

---

✅ Node automatically looks for:

index.js

when you import a folder

👉 So this works:

JavaScript

```
require('./controllers')
```

instead of:

JavaScript

```
require('./controllers/index.js')
```

## Interview Questions

---

- 👉 Why use `index.js` in a folder?
- 👉 How does Node resolve directory imports?
- 👉 Difference between:



JavaScript

```
require('./folder')  
vs  
require('./folder/index.js')
```

## Real Project Example (MERN)

---

```
routes/  
controllers/  
models/  
middlewares/
```

Each folder has:

```
index.js
```

👉 Makes project:

- ✓ Clean
- ✓ Scalable
- ✓ Easy to import

## 🔥 Pro Tip

---

Always structure like:

```
JavaScript
```

```
module.exports = {  
  feature1,  
  feature2  
};
```

👉 Makes your backend **professional level** 100

If you want next level 🚀

I can:

- 👉 Show **complete backend folder structure (industry level)**
- 👉 Build **Express project using directory exports**
- 👉 Or explain **require() internally कैसे काम करता है**

what is npm

Let's understand **npm** in a simple but complete way 👉

## 📦 What is npm?

---

👉 **npm (Node Package Manager)** is a tool that comes with Node.js

👉 It is used to:

- Install packages (libraries)
- Manage dependencies
- Run scripts

📌 It is the **world's largest software registry**

## 🧠 Simple Definition

---

👉 npm is like a **Play Store for Node.js packages**

## Check npm Version

---

Bash

```
npm -v
```

## Install a Package

---

### ◆ Local Install (project only)

---

Bash

```
npm install express
```

## ◆ Global Install

---

Bash

```
npm install -g nodemon
```

## 📁 package.json (Very Important)

---

👉 Created using:

Bash

```
npm init -y
```

📌 Example:

JSON

```
{  
  "name": "my-app",  
  "version": "1.0.0",  
  "dependencies": {  
    "express": "^4.18.2"  
  }  
}
```



## Install All Dependencies

---

Bash

```
npm install
```

👉 Installs everything from `package.json`

## Update Package

---

Bash

```
npm update
```

## Remove Package

---

Bash

```
npm uninstall express
```

## node\_modules Folder

---

👉 Stores all installed packages

🚫 Don't upload this to GitHub  
(use `.gitignore`)

## package-lock.json

---

👉 Locks exact versions

👉 Ensures same install everywhere

## npm Scripts

---

👉 Add in `package.json`:

JSON

```
"scripts": {  
  "start": "node app.js",  
  "dev": "nodemon app.js"  
}
```

👉 Run:

Bash

```
npm run dev
```

 **Search Packages**

---

👉 Visit:

- npm website
- or use:

```
Bash
```

```
npm search express
```

## Popular npm Packages

---

- express → backend framework
- mongoose → MongoDB
- nodemon → auto restart
- dotenv → environment variables
- cors → API security

## Common Errors

---

### npm not recognized

---

 Fix:

- Install Node.js properly
- Restart terminal

## Interview Questions

---

-  What is npm?
-  Difference between local & global install?
-  What is package.json?
-  What is node\_modules?



## Real Example

---

Bash

```
npm init -y  
npm install express
```

JavaScript

```
const express = require('express');  
const app = express(); app.listen(3000);
```

## Pro Tips

---

 Always run:

Bash

```
npm init -y
```

✓ Use:

Bash

```
npm install package_name
```

✓ Use `.gitignore` for:

Bash

```
node_modules/
```

If you want next 🚀

I can explain:

👉 **npx vs npm**

👉 OR build a **project using npm + Express step-by-step**

👉 OR give **top 50 npm interview questions**

installing packages

Let's learn **how to install packages in Node.js using npm** step-by-step 👉

## 📦 What is Installing a Package?

👉 Installing a package means **downloading a library from npm** and using it in your project.

## Step 1: Initialize Project (First Time Only)

---

Bash

```
npm init -y
```

👉 This creates:

Bash

```
package.json
```

## Step 2: Install a Package

---

### ◆ Install Locally (Most Common)

---

Bash

```
npm install express
```

👉 Shortcut:

Bash

```
npm i express
```

📌 This will:

- Create `node_modules/`
- Update `package.json`
- Create `package-lock.json`

### 🌐 Step 3: Use Installed Package

---

JavaScript

```
const express = require('express'); const app = express();  
app.listen(3000);
```

## Install Multiple Packages

---

Bash

```
npm install express mongoose cors
```

## Install Globally

---

Bash

```
npm install -g nodemon
```

👉 Use anywhere:

Bash

```
nodemon app.js
```



## Install Specific Version

---

Bash

```
npm install express@4.18.2
```



## Install Dev Dependencies

---

👉 For development only:

Bash

```
npm install nodemon --save-dev
```

👉 In `package.json`:

JSON

```
"devDependencies": {  
  "nodemon": "^3.0.0"  
}
```

## Install All Dependencies

---

👉 If project already exists:

Bash

```
npm install
```

## Uninstall Package

---

Bash

```
npm uninstall express
```

## Check Installed Packages

---

```
Bash
```

```
npm list
```

## Common Errors & Fix

---

### npm not recognized

---

 Fix:

- Install Node.js properly
- Restart terminal

### Permission error (Linux/Mac)

---

```
Bash
```

```
sudo npm install
```

### Wrong place

---

 Make sure you're inside project folder:



Bash

```
cd project-name
```



## Best Practices

---

✓ Always run:

Bash

```
npm init -y
```

✓ Use:

Bash

```
npm install package_name
```

✓ Add to `.gitignore`:

Bash

node\_modules/

## Real Example (Full Flow)

---

Bash

```
mkdir myapp
cd myapp
npm init -y
npm install express
```

JavaScript

```
// app.js
const express = require('express');
const app = express(); app.listen(3000, () => {
  console.log("Server running");
});
```

```
Bash
```

```
node app.js
```

## Pro Tip

---

👉 Use this for faster install:

```
Bash
```

```
npm i
```

If you want next 🔥

I can teach:

- 👉 **npx vs npm (very important)**
- 👉 **OR how packages work internally**
- 👉 **OR build a real backend project step-by-step**

local vs global installation

Let's clearly understand **Local vs Global Installation in npm** (very important concept 👉)

## 1. Local Installation

---

👉 Installed **inside your project folder**

## ✓ Command:

---

Bash

```
npm install express
```

## 📌 Stored in:

Bash

```
node_modules/
```

## 📌 Saved in:

Bash

```
package.json
```

## ◆ Usage:

---

👉 Used **only inside that project**

JavaScript

```
const express = require('express');
```

### ✓ Example:

---

Bash

```
npm install express
```

✓ Used for:

- Libraries (express, mongoose, cors)
- Project dependencies

## 2. Global Installation

---

👉 Installed **on your system (computer)**

## ✓ Command:

---

Bash

```
npm install -g nodemon
```

## ◆ Usage:

---

👉 Can be used **anywhere in terminal**

Bash

```
nodemon app.js
```

## ✓ Example:

---

Bash

```
npm install -g nodemon
```

✓ Used for:

- Tools / CLI utilities
- Development tools

## Key Differences

---

| Feature      | Local Install  | Global Install |
|--------------|----------------|----------------|
| Location     | Inside project | System-wide    |
| Command      | npm install    | npm install -g |
| Access       | Only project   | Everywhere     |
| Stored in    | node_modules   | System folder  |
| package.json | Yes            | No             |

## Easy Trick to Remember

---

👉 Local = **Project use**

👉 Global = **System tool**

## Real Example

---

✓ Local:

---

Bash

```
npm install express
```

👉 Used in code:

JavaScript

```
require('express');
```

## ✓ Global:

---

Bash

```
npm install -g nodemon
```

👉 Used in terminal:

Bash

```
nodemon app.js
```

## ⚠ Common Mistake

---

✗ Installing everything globally

👉 Wrong ✗



✓ Correct:

- Libraries → Local
- Tools → Global

## Interview Questions

---

- 👉 Difference between local & global install?
- 👉 Why express is local but nodemon is global?
- 👉 Can we use global package in project?

## Pro Tips

---

✓ Always install:

Bash

```
npm install express  ✓ local
```

✓ Use global only for tools:

Bash

```
npm install -g nodemon
```

If you want next 🔥

I can explain:

👉 **npx vs global install (very important concept)**

- 👉 OR **package.json** deep explanation
- 👉 OR give **real interview questions + answers**

import module

Let's understand **how to import modules in Node.js** (very important concept) 📌

## 🧠 What is Importing a Module?

---

👉 Import means **using code from another file or package**

## 📦 1. CommonJS (Default in Node.js)

---

👉 Uses:

- `require()` → import
- `module.exports` → export

### ◆ Import Local File

---

👉 **math.js**

---

JavaScript

```
exports.add = (a, b) => a + b;
```

JavaScript

```
const math = require('./math'); console.log(math.add(2, 3));
```

## ◆ Import Specific Function

---

JavaScript

```
const { add } = require('./math'); console.log(add(5, 3));
```

## ◆ Import JSON File

---

 **data.json**

JSON

```
{  
  "name": "Aamir"  
}
```

👉 app.js

JavaScript

```
const data = require('./data.json');  
console.log(data.name);
```

## ◆ Import Built-in Module

---

JavaScript

```
const fs = require('fs'); fs.writeFileSync("file.txt", "Hello");
```

## ◆ Import Third-party Module

---

👉 First install:

Bash

```
npm install express
```

👉 Then:

JavaScript

```
const express = require('express');
```

## 🌐 2. ES Modules (Modern Way)

---

👉 Uses:

- `import` → import
- `export` → export

## ⚙️ Enable ES Modules

---

👉 In `package.json`:

JSON

```
{  
  "type": "module"  
}
```

## ◆ Import File

---

👉 **math.js**

---

JavaScript

```
export const add = (a, b) => a + b;
```

👉 **app.js**

---

JavaScript

```
import { add } from './math.js'; console.log(add(2, 3));
```

## ◆ Default Import

---

JavaScript

```
// math.js
export default function greet() {
  console.log("Hello");
}
```

JavaScript

```
// app.js
import greet from './math.js';
```

## Important Rules

---

### 1. Local file must have `./`

---

JavaScript

```
require('./math') ✓  
require('math') ✗
```

### 2. ES Modules need extension

---

JavaScript

```
import './math.js' ✓
```



## Common Errors

---

### Error:

---

JavaScript

Cannot use import outside module

#### ✓ Fix:

- Add `"type": "module"`  
OR
- Use `.mjs`

### Error:

---

JavaScript

module not found

#### ✓ Fix:

- Check file path
- Use correct `./`

## require vs import

---

| Feature         | require() | import       |
|-----------------|-----------|--------------|
| Type            | CommonJS  | ES Module    |
| Sync/Async      | Sync      | Async        |
| Default in Node | Yes       | Needs config |

## Interview Questions

---

- 👉 Difference between `require` and `import`
- 👉 Can we mix both?
- 👉 What is default export?

## Real Example (Project)

---

JavaScript

```
// utils/math.js
exports.add = (a, b) => a + b; // app.js
const { add } = require('./utils/math'); console.log(add(10, 5));
```

## Pro Tips

---

✅ Use:

JavaScript

```
const { functionName } = require('./file');
```

- ✓ Keep clean folder structure
- ✓ Prefer ES Modules in modern projects

If you want next 🔥

I can teach:

- 👉 **require()** internally कैसे काम करता है
- 👉 OR **full Express project using modules**
- 👉 OR **advanced module patterns (interview level)**

which command to use for in the file

Your question is a bit unclear, but I think you're asking:

👉 **"Which command should I use inside a file to import modules in Node.js?"**

Let's make it crystal clear 👉

## 🧠 Which Command to Use in File?

---

👉 It depends on **which module system you are using**:

### 📦 1. Default Node.js → Use **require()**

---

- ✓ Most common (especially in your learning phase)

JavaScript

```
const express = require('express');
```

👉 For local file:

JavaScript

```
const math = require('./math');
```

## 2. Modern JavaScript → Use **import**

---

👉 Only if ES Modules enabled

JavaScript

```
import express from 'express';
```

👉 For local file:

JavaScript

```
import { add } from './math.js';
```

## When to Use What?

---

### Use **require()** when:

---

- No **"type": "module"** in package.json
- Working with traditional Node.js
- Most tutorials/projects (like yours)

### Use **import** when:

---

You added in **package.json**:

JSON

```
{  
  "type": "module"  
}
```

Using modern JS (ES6+)

## Important Rule

---

 Don't mix both in same file:

JavaScript

`require(...)` ❌  
`import ...` ❌ together

## 🎯 Simple Answer (Interview Style)

---

👉 Q: Which command is used to import modules in Node.js?

👉 A:

- `require()` in CommonJS
- `import` in ES Modules

## 🔥 Recommendation (For You)

---

👉 Since you're learning backend (Express + MySQL):

✓ Use:

JavaScript

```
const express = require('express');
```

👉 Later switch to `import`

If you want next 🚀

I can show:

👉 Full **file structure with imports** (industry level)

- 👉 OR explain **require()** internally कैसे काम करता है
- 👉 OR build your **Zerodha backend step-by-step**

which command to use for enter the file

Got it 👍 — you're asking:

👉 “Which command is used to enter/open a file?”

It depends on what you mean by “enter” 👉

## 📁 1. Enter a Folder (Directory)

---

👉 Use:

```
Bash
```

```
cd folder-name
```

## ✅ Example:

---

```
Bash
```

```
cd project
```

## 📄 2. Open a File in Terminal (View Content)

---

👉 Use:

Bash

```
type filename    (Windows)
cat filename      (Mac/Linux)
```



### Example:

---

Bash

```
type app.js
```



### 3. Open File in VS Code (Best Way)

---



Use:

Bash

```
code filename
```



## ✓ Example:

---

Bash

```
code app.js
```

👉 Open full folder:

Bash

```
code .
```

## ▶ 4. Run a Node File (Most Important for You)

---

👉 Use:

Bash

```
node filename
```

## ✓ Example:

---

Bash

```
node app.js
```

## 5. Create a File

---

👉 Windows:

Bash

```
type nul > app.js
```

👉 Or VS Code:

Bash

```
code app.js
```

## ⚠ Common Mistake (You Did Earlier)

---

✗ Writing:

```
JavaScript
```

```
node app.js
```

inside browser console

✓ Correct:

👉 Run it in **terminal (CMD / PowerShell / VS Code terminal)**

## 🎯 Quick Summary

---

| Task         | Command                   |
|--------------|---------------------------|
| Enter folder | <code>cd folder</code>    |
| Open file    | <code>code file</code>    |
| Run file     | <code>node file.js</code> |
| View file    | <code>type file</code>    |

## Best Practice (Use This Flow)

---

Bash

```
cd myproject
code .
node app.js
```

If you want next 🔥

I can teach:

- 👉 **All terminal commands (complete guide)**
- 👉 OR fix your environment step-by-step (why node errors came earlier)
- 👉 OR build a **Node project from scratch**

import module

Got it 👍 — let's **directly answer “import module” in Node.js** in the simplest and correct way.

## Import Module in Node.js

---

👉 To import a module, you use:

## 1. Using `require()` (Most Common)

---

### Syntax:

---

JavaScript

```
const moduleName = require('module-name');
```

### ◆ Example 1: Built-in Module

---

JavaScript

```
const fs = require('fs'); fs.writeFileSync("test.txt", "Hello");
```

### ◆ Example 2: Local File

---

 math.js

JavaScript

```
exports.add = (a, b) => a + b;
```

👉 app.js

JavaScript

```
const math = require('./math'); console.log(math.add(2, 3));
```

### ◆ Example 3: Specific Import

---

JavaScript

```
const { add } = require('./math'); console.log(add(5, 3));
```

### ◆ Example 4: Third-party Module

---

👉 Install first:

Bash

```
npm install express
```

👉 Use:

JavaScript

```
const express = require('express');
```

## 2. Using **import** (Modern Way)

---

👉 Only works if ES Modules enabled

### Step 1: Enable

---

👉 package.json:

JSON

```
{  
  "type": "module"  
}
```

### ◆ Example:

---

👉 math.js

JavaScript

```
export const add = (a, b) => a + b;
```

👉 app.js

JavaScript

```
import { add } from './math.js'; console.log(add(2, 3));
```



## ◆ Default Import

---

JavaScript

```
// math.js
export default function greet() {
  console.log("Hello");
} // app.js
import greet from './math.js';
```

## ⚠ Important Rules

---

### ✓ For local files:

---

JavaScript

```
require('./file') ✓
```

## Wrong:

---

JavaScript

```
require('file') 
```

## ES Modules need extension:

---

JavaScript

```
import './file.js'
```

## Common Errors

---

### Error:

---

JavaScript

```
module is not defined
```

👉 Using `require` in browser ❌

❌ **Error:**

---

```
JavaScript
```

```
Cannot use import outside module
```

👉 Add `"type": "module"`

🎯 **Final Simple Answer**

---

👉 To import module:

✓ In Node (default):

```
JavaScript
```

```
const moduleName = require('module-name');
```

✓ In modern JS:

JavaScript

```
import moduleName from 'module-name';
```

## Best Recommendation (For You)

---

👉 Use this for now:

JavaScript

```
const express = require('express');
```

👉 Because:

- You're learning backend
- Most Node projects still use it

If you want next 🔥

I can:

- 👉 Show **full project using modules (Express + MySQL)**
- 👉 OR explain **require() internally कैसे काम करता है**
- 👉 OR give **interview questions on modules**

export file and directories with all concept

Alright — let's cover **Export in Files & Directories (FULL Node.js concept)** from **basic** → **advanced** → **interview level** in one place 

## 1. What is Export?

---

👉 Export = share code from one file to another

- ✓ Functions
- ✓ Variables
- ✓ Objects
- ✓ Classes

## 2. Export in Files (CommonJS)

---

### ◆ Method 1: `module.exports` (Best)

---

👉 `math.js`

---

JavaScript

```
const add = (a, b) => a + b;  
const sub = (a, b) => a - b; module.exports = { add, sub };
```

👉 `app.js`

---

JavaScript

```
const math = require('./math'); console.log(math.add(5, 3));
```

## ◆ Method 2: exports

---

JavaScript

```
exports.add = (a, b) => a + b;  
exports.sub = (a, b) => a - b;
```

## ⚠ Important Difference

---

✗ Wrong:

JavaScript

```
exports = { add, sub };    // ✗ breaks reference
```

✓ Correct:

JavaScript

```
module.exports = { add, sub };
```

## ◆ Export Single Value

---

JavaScript

```
module.exports = function(name) {  
  return "Hello " + name;  
};
```

## ◆ Export Class

---

JavaScript

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
} module.exports = User;
```

### 3. ES Modules (Modern Export)

---

#### Enable:

---

JSON

```
{  
  "type": "module"  
}
```

#### ◆ Named Export

---

JavaScript

```
export const add = (a, b) => a + b;  
export const sub = (a, b) => a - b;
```



## ◆ Default Export

---

JavaScript

```
export default function greet() {  
  console.log("Hello");  
}
```

## ◆ Import

---

JavaScript

```
import greet from './file.js';  
import { add } from './file.js';
```

## 📁 4. Export in Directories (VERY IMPORTANT)

---

👉 Used in real projects (Express, MVC)



## Folder Structure

---

```
project/
├── controllers/
│   ├── user.js
│   ├── product.js
│   └── index.js    👉 main export file
└── app.js
```

### ◆ Step 1: Individual Files

---

#### 👉 user.js

---

JavaScript

```
exports.getUser = () => "User Data";
```

## product.js

---

JavaScript

```
exports.getProduct = () => "Product Data";
```

## ◆ Step 2: Export from Directory

---

## controllers/index.js

---

JavaScript

```
const user = require('./user');
const product = require('./product'); module.exports = {
  user,
  product
};
```

### ◆ Step 3: Import Directory

---

JavaScript

```
const controllers = require('./controllers'); controllers.user.getUser();  
controllers.product.getProduct();
```

## 🔥 5. Advanced Directory Export (BEST PRACTICE)

---

👉 controllers/index.js

JavaScript

```
module.exports = {  
  ...require('./user'),  
  ...require('./product')  
};
```

👉 Use directly:

JavaScript

```
const { getUser, getProduct } = require('./controllers'); getUser();  
getProduct();
```

## 6. ES Module Directory Export

---

JavaScript

```
export * from './user.js';  
export * from './product.js';
```

## 7. How Node Resolves Directory Import

---

 When you write:

JavaScript

```
require('./controllers')
```

👉 Node automatically looks for:

```
controllers/index.js
```

## ⚠️ 8. Common Errors

---

### ❌ Module not found

---

👉 Fix:

- Check path (./)
- Check file name

### ❌ exports not working

---

👉 Fix:

```
JavaScript
```

```
module.exports = {} ✓
```

### ❌ Cannot use import outside module

---

👉 Fix:

Add "type": "module"

## 9. Real Project Structure (Industry)

---

```
src/  
|  
├─ controllers/  
|   └─ index.js  
├─ routes/  
|   └─ index.js  
├─ models/  
|   └─ index.js  
├─ services/  
|   └─ index.js
```

## 10. Interview Questions

---

👉 Difference:

`exports` vs `module.exports`

- 👉 What happens when importing a directory?
- 👉 How Node resolves modules?
- 👉 Default vs named export?

## 11. Pro Tips (Very Important)

---

✅ Always use:

```
JavaScript
```

```
module.exports = {}
```

✅ Use `index.js` in every folder

✓ Use destructuring:

JavaScript

```
const { functionName } = require('./folder');
```

✓ Keep code modular

## 🔥 Final Summary

---

👉 File export:

JavaScript

```
module.exports = { data }
```

👉 Directory export:



JavaScript

```
module.exports = {  
  ...require('./file1'),  
  ...require('./file2')  
}
```

👉 Import:

JavaScript

```
const data = require('./folder');
```

If you want next 🚀

I can:

- 👉 Build **complete backend (Express + MySQL) using this structure**
- 👉 OR explain **require() internal working (deep concept)**
- 👉 OR give **top 50 interview questions with answers**

require vs import

Let's clearly understand **require vs import in Node.js** (very important for interviews + real projects) 👉

## 🧠 Basic Difference

---

👉 Both are used to **import modules**, but they belong to different systems:

- **require** → **CommonJS (old/default Node.js)**

- `import` → ES Modules (modern JavaScript)

## 1. `require()` (CommonJS)

---

### Syntax:

---

JavaScript

```
const express = require('express');
```

### ◆ Features:

---

- Synchronous (loads immediately)
- Default in Node.js
- No config needed

### ◆ Example:

---

JavaScript

```
// math.js
exports.add = (a, b) => a + b; // app.js
const { add } = require('./math'); console.log(add(2, 3));
```

## 2. **import** (ES Modules)

---

### **Syntax:**

---

JavaScript

```
import express from 'express';
```

### **Enable ES Modules:**

---

JSON

```
{  
  "type": "module"  
}
```

## ◆ Example:

---

JavaScript

```
// math.js
export const add = (a, b) => a + b; // app.js
import { add } from './math.js'; console.log(add(2, 3));
```

## Key Differences

---

| Feature         | <b>require()</b>      | <b>import</b>           |
|-----------------|-----------------------|-------------------------|
| Module system   | CommonJS              | ES Modules              |
| Loading         | Synchronous           | Asynchronous            |
| Default in Node | Yes                   | No (needs config)       |
| Syntax          | require()             | import                  |
| File extension  | Not needed            | Required ( <b>.js</b> ) |
| Usage           | Backend (traditional) | Modern JS               |

## Important Rules

---

### Don't mix:

---

JavaScript

`require()`   
`import` 

### Error:

---

JavaScript

Cannot use import outside module

### ✓ Fix:

- Add `"type": "module"`  
OR
- Use `.mjs`

## When to Use What?

---

### ✓ Use `require()` when:

---

- Learning Node.js (your case)

- Using older projects
- Quick backend setup

### ✓ Use **import** when:

---

- Modern projects (React, ES6)
- Clean & scalable code
- Using latest Node.js

### 🎯 Interview Answer (Short)

---

👉 **require()** is used in CommonJS and works synchronously, while **import** is used in ES Modules and works asynchronously.

### 🚀 Real Example (Both)

---

#### CommonJS:

---

JavaScript

```
const fs = require('fs');
```

## ES Module:

---

JavaScript

```
import fs from 'fs';
```

### Pro Tip

---

👉 For you right now:

✓ Use:

JavaScript

```
const express = require('express');
```

👉 Later upgrade to:

JavaScript

```
import express from 'express';
```

## 🔥 Final Conclusion

---

👉 `require` = Simple + default

👉 `import` = Modern + future

If you want next 🚀

I can:

👉 Explain **how `require` works internally (deep concept)**

👉 OR convert your project from `require` → `import`

👉 OR give **advanced interview questions on modules**

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#).